Task 4: Mini SQL Agent
(Agentic System Task)

Goal

To build a simple **agentic system that thinks, acts, and corrects itself.** A working **AI-powered SQL agent system** that mimics real production-level data assistants like analytics chatbots and BI tools.

GitHub Repo:(Folder: “task4”)

https://github.com/nilima-sth/text-to-sql-agent-sys.git

Sheet:

(Sheet Four:* Task_4_Gen_sql*)
(Sheet Five:* Evaluation_for_task4*)
Task_4_Generated_sql

Required Agent Flow

Step 1: Understand Query

- Identify intent
- Detect tables and columns

Step 2: Generate SQL

- Write SQL query based on structure

Step 3: Execute Query

- Run SQL in PostgreSQL

Step 4: Error Handling

If query fails:

- Read error message
- Fix query
- Retry up to **3 times maximum**

Step 5: Final Output

- Return SQL result
- Convert result into natural language summary

Evaluation Criteria

Understanding

- Correct decomposition of queries

Implementation

- Working SQL generation
- Correct API structure

Agent Behavior

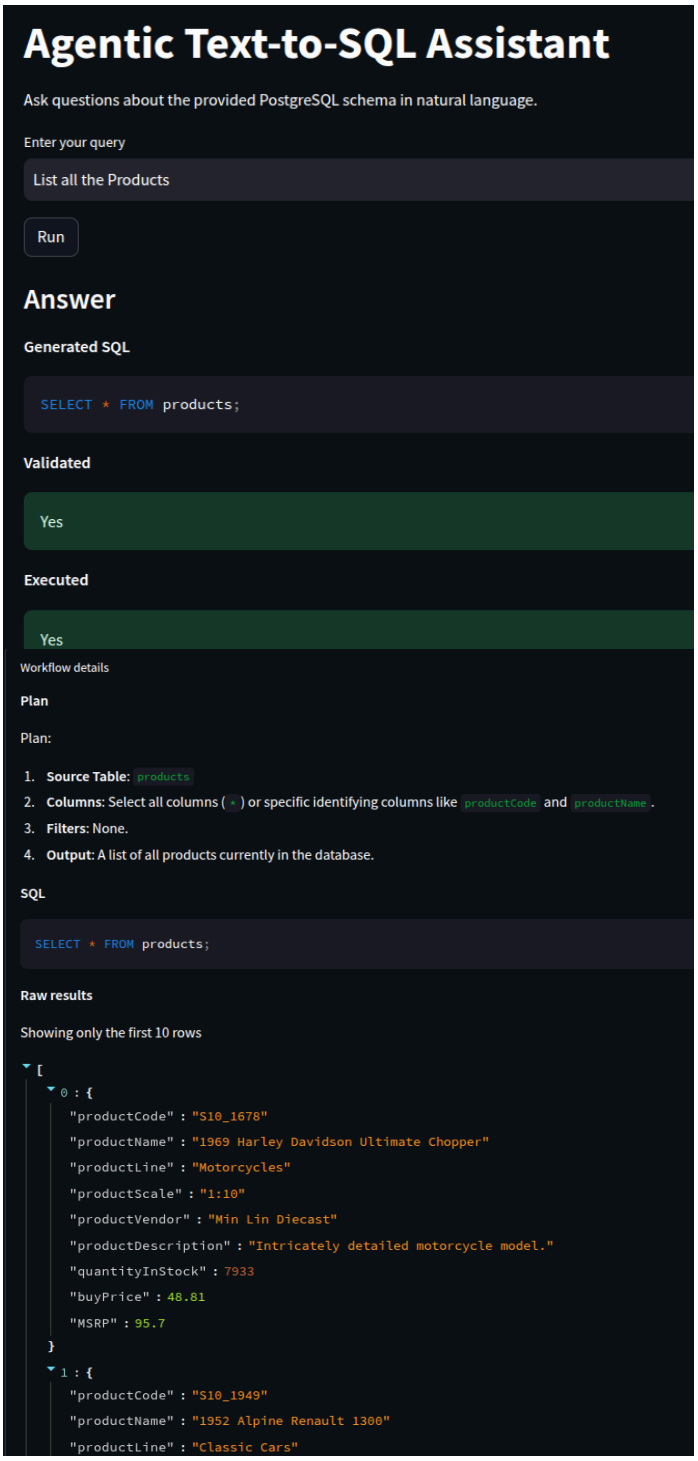
- Retry mechanism
- Error handling

System Design

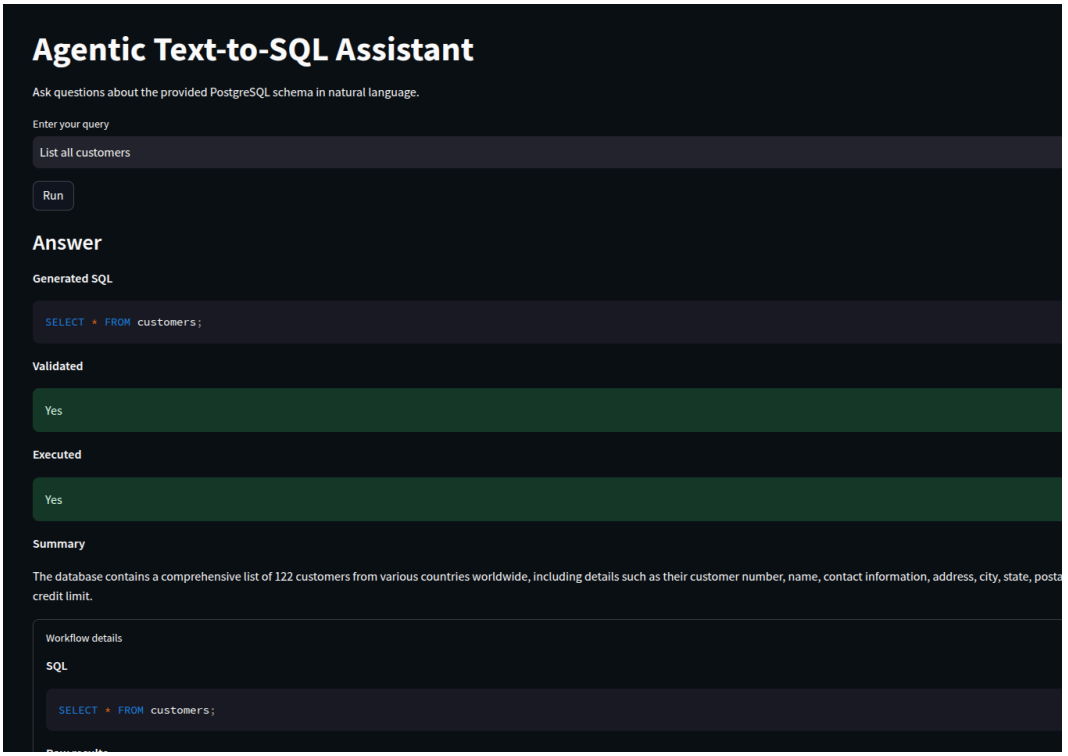
- Clean FastAPI structure
- Logging implementation
- Safe SQL execution

SOME EXAMPLES (SCREENSHOTS)

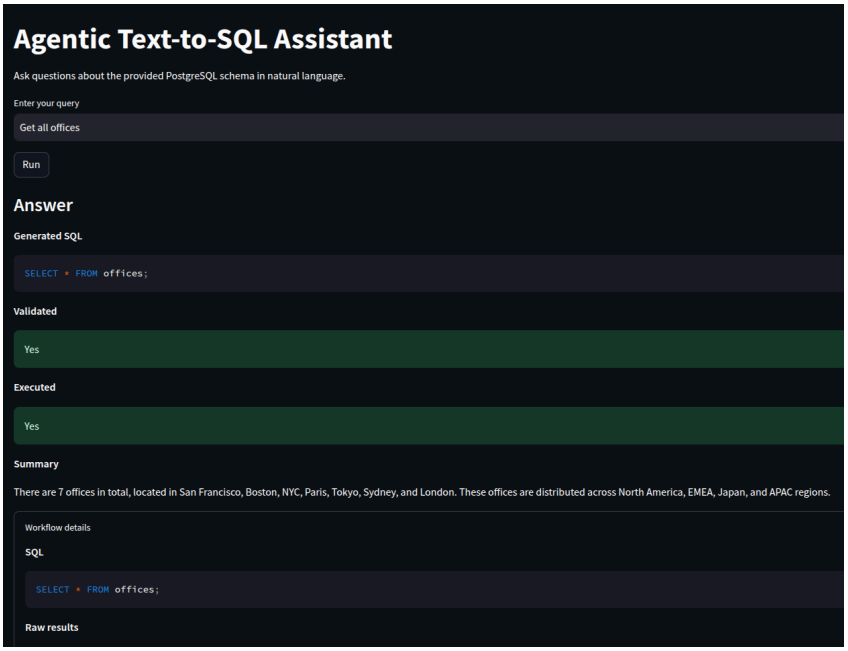
Example 1: List all the products



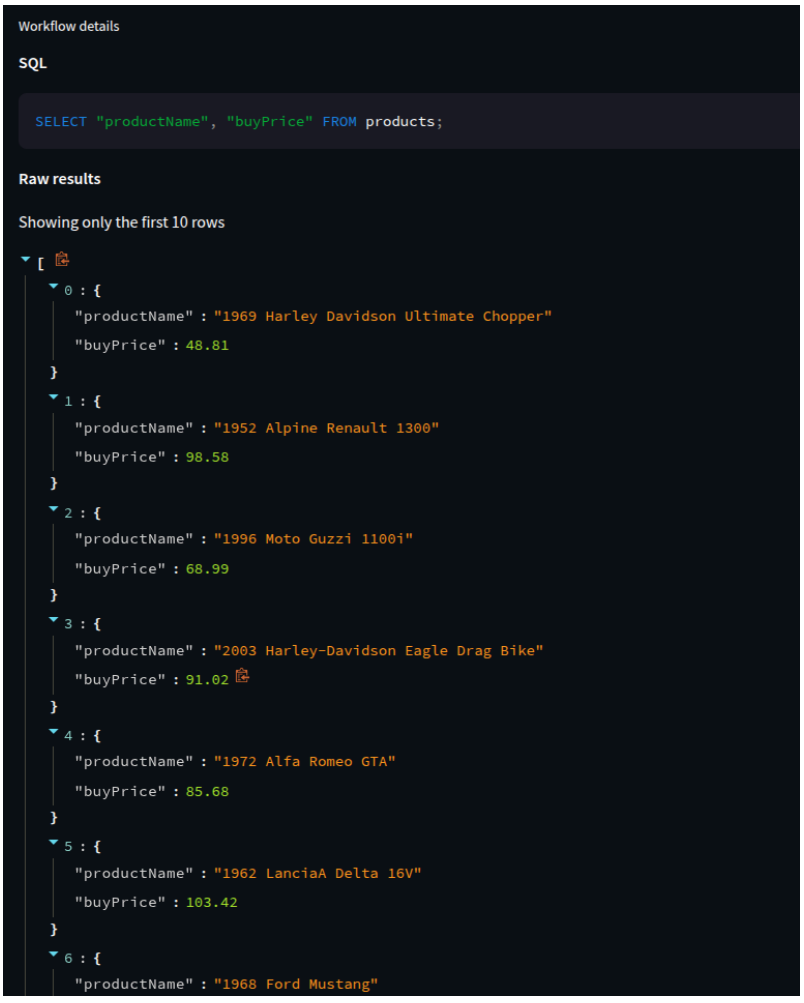
Example 2: Get all customers



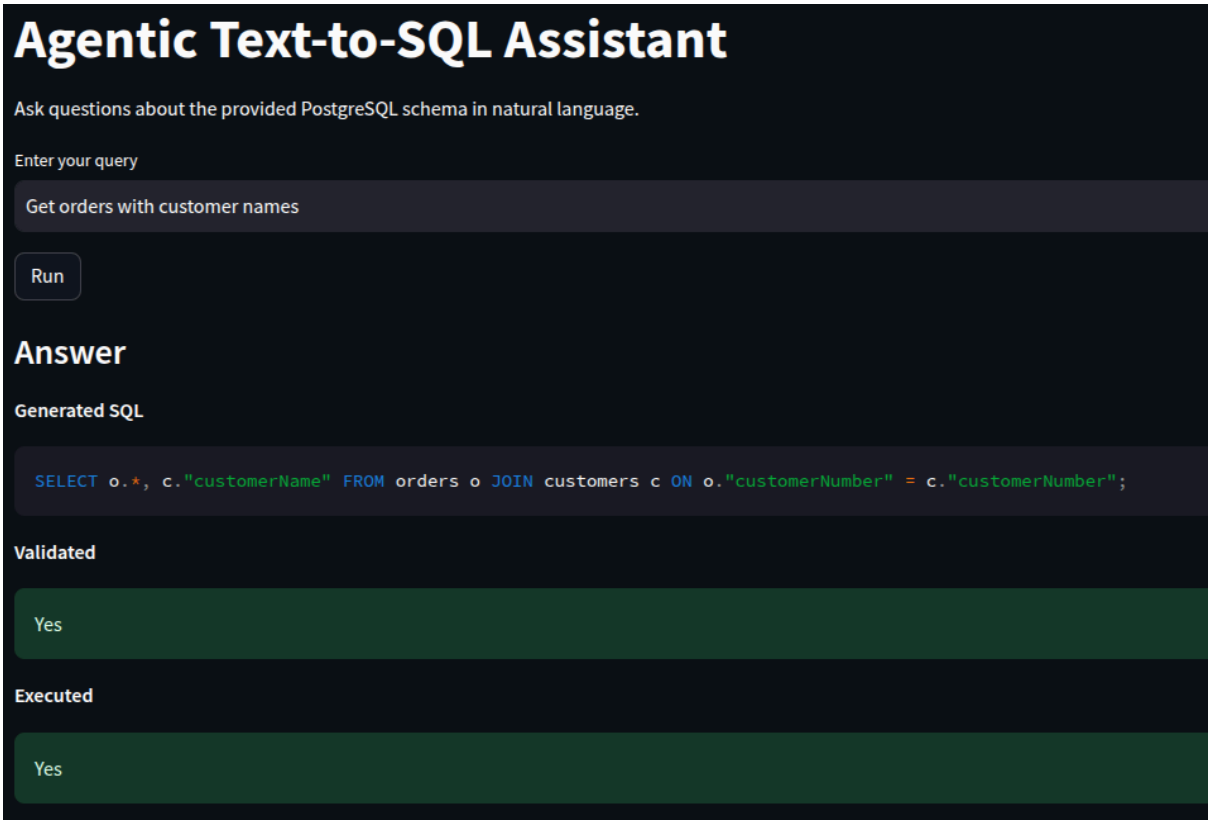
Example 3: Get all offices

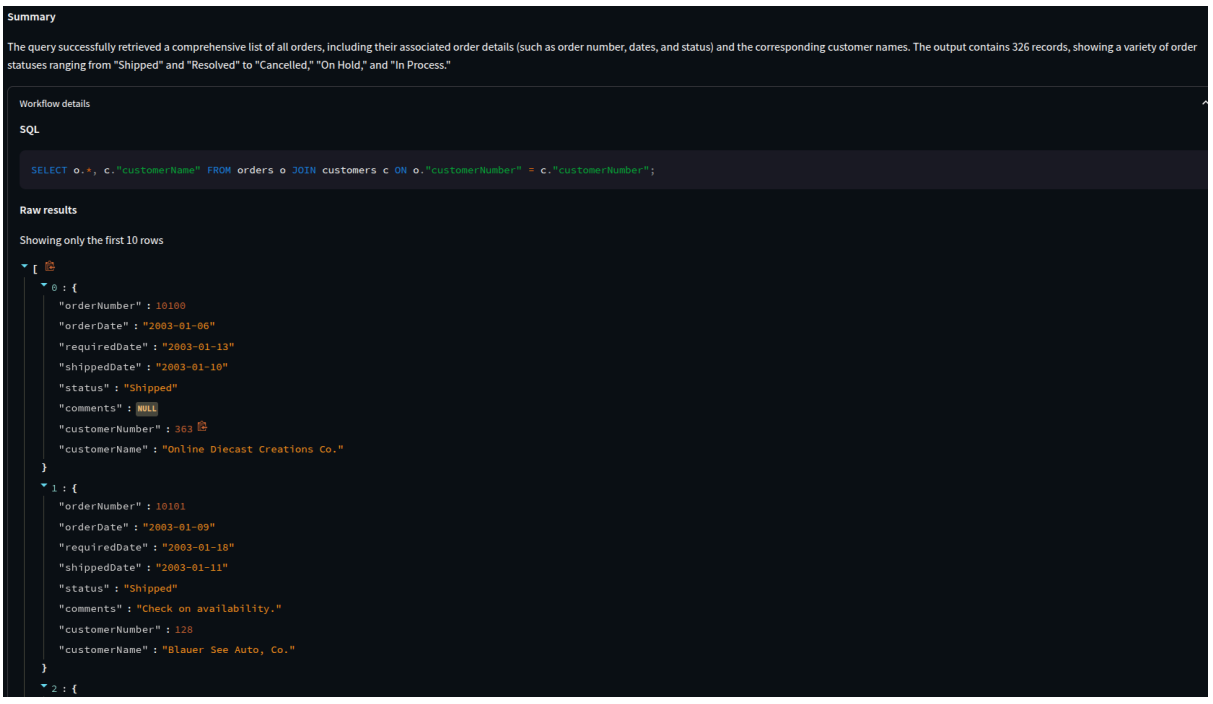


Example 4: Get product names and prices

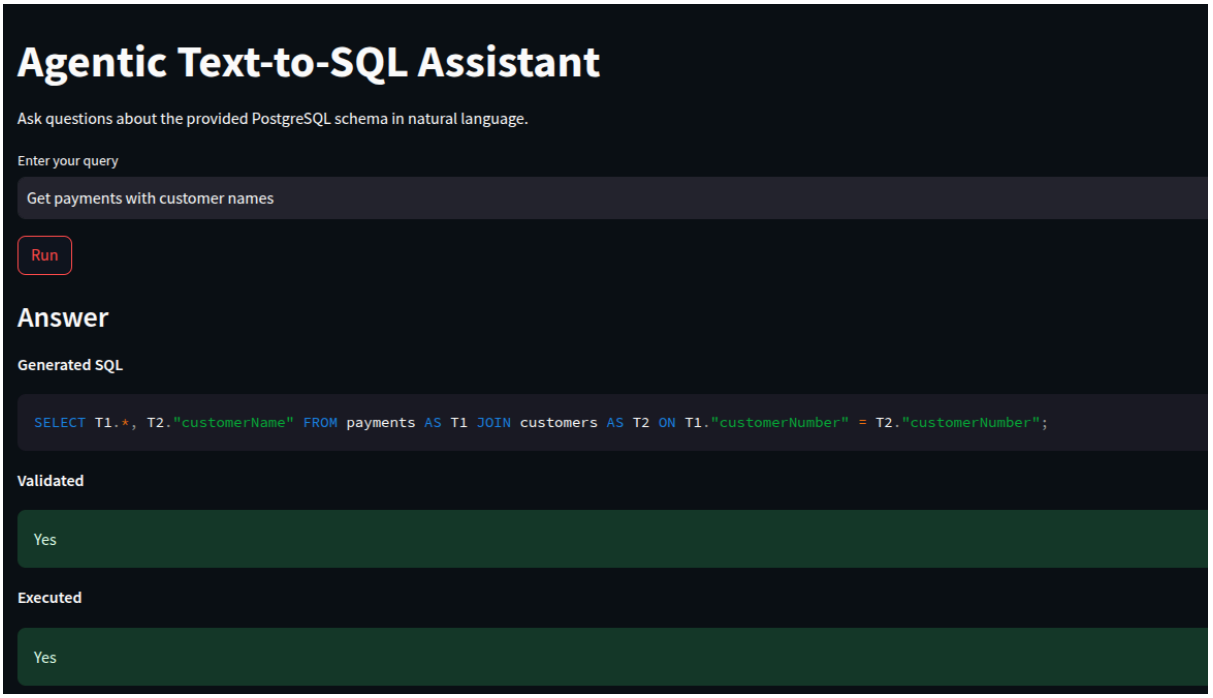


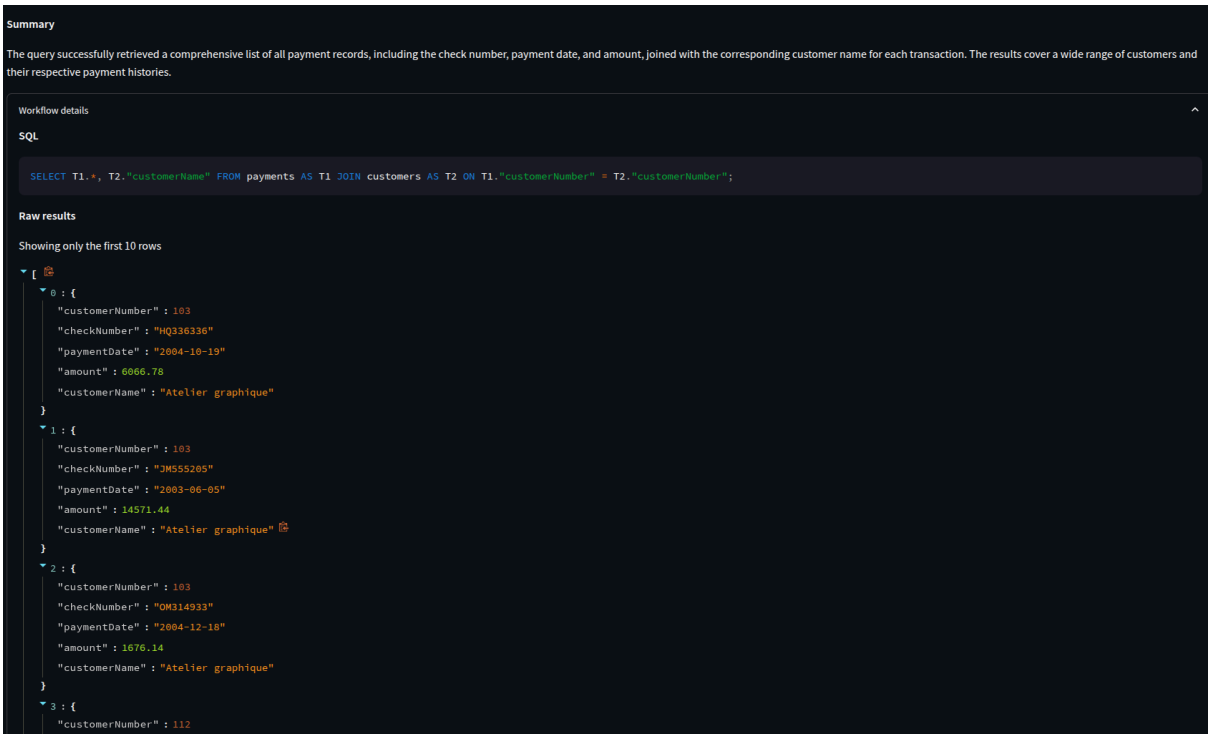
Example 5: Get orders with customer names



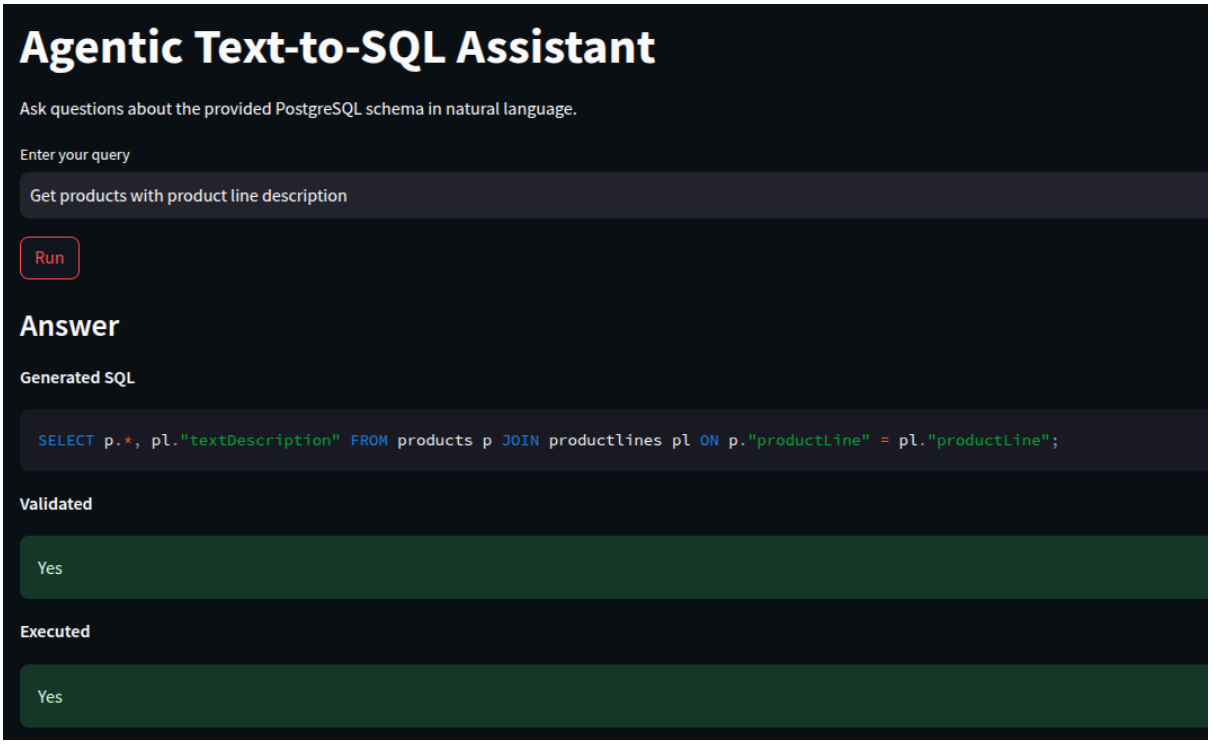


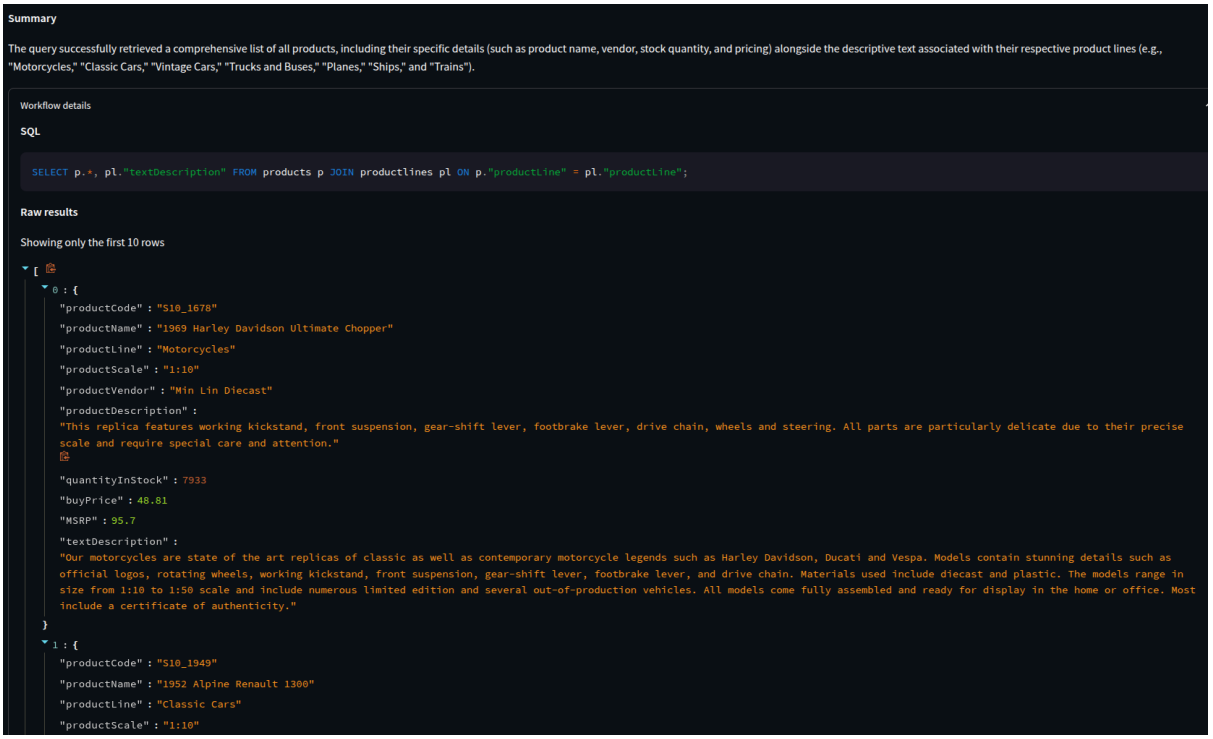
Example 6: Get payments with customer names



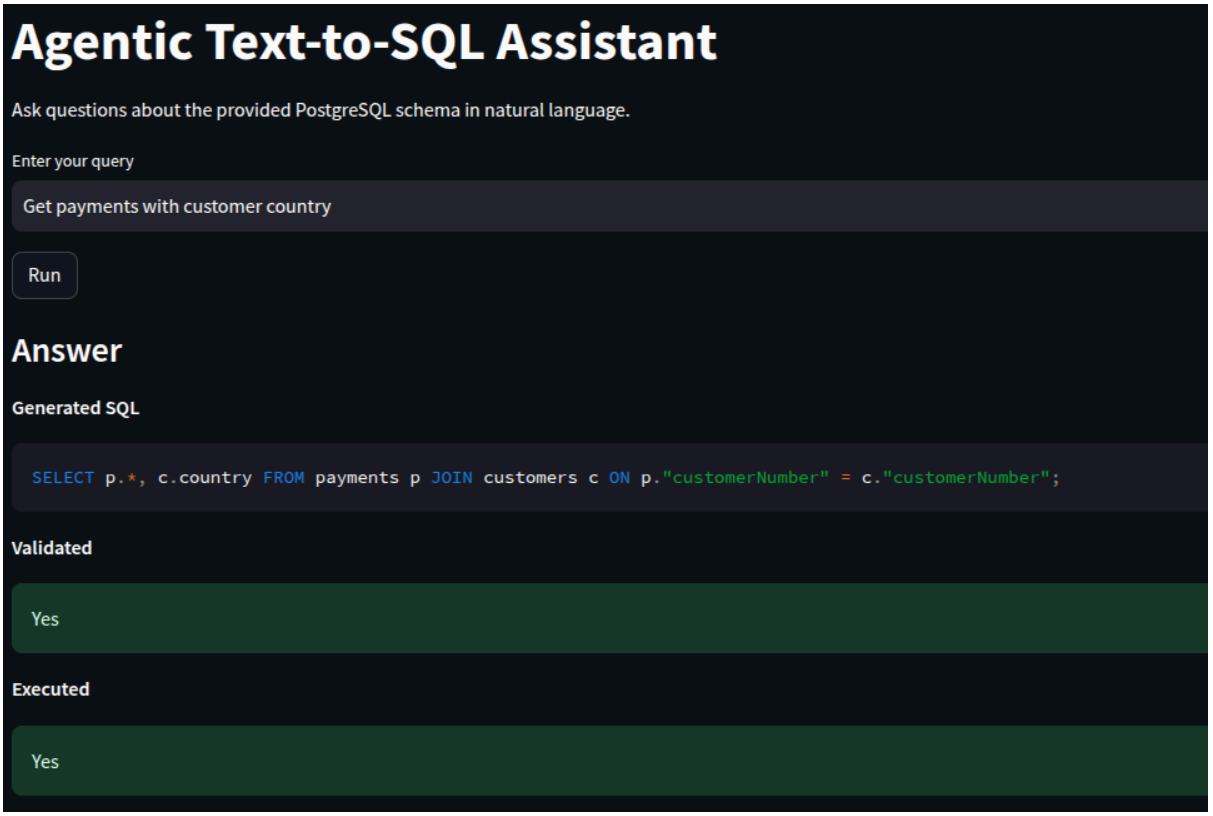


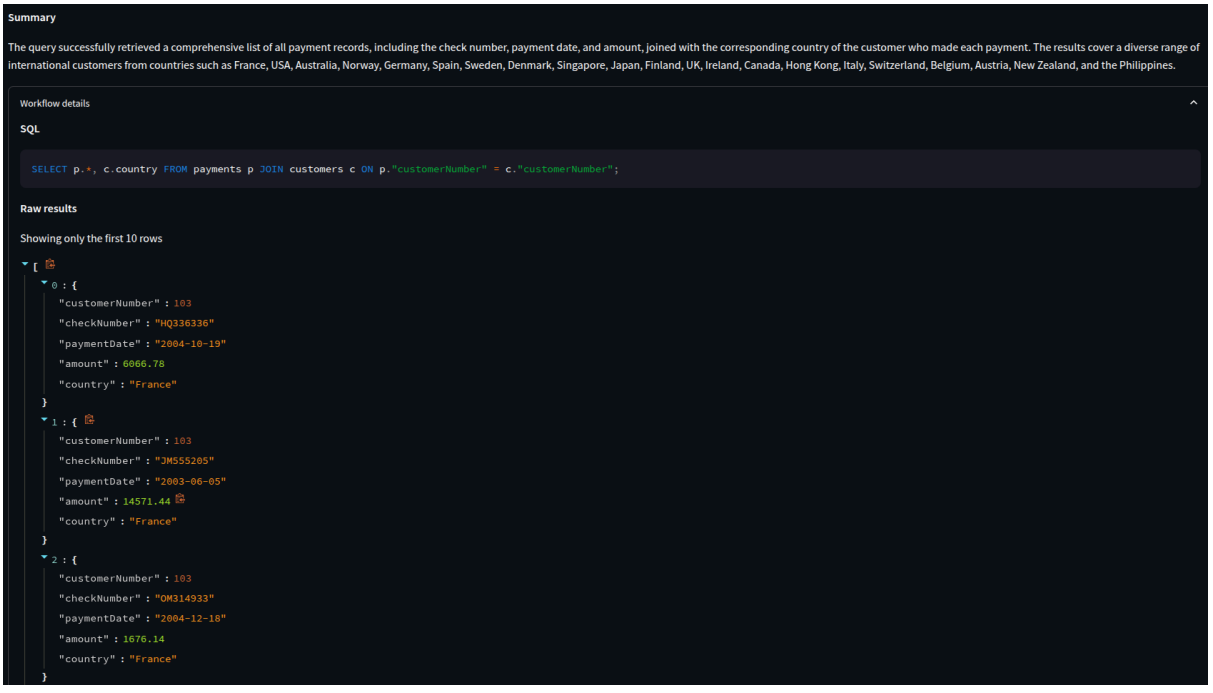
Example 7: Get products with product line description



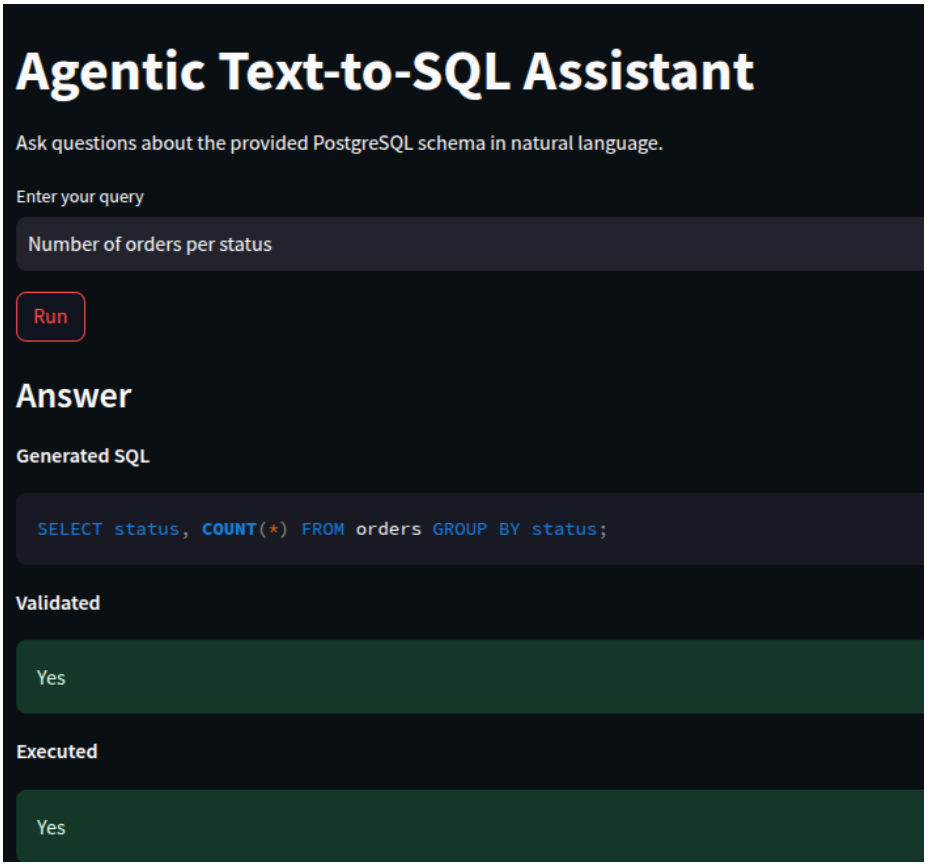


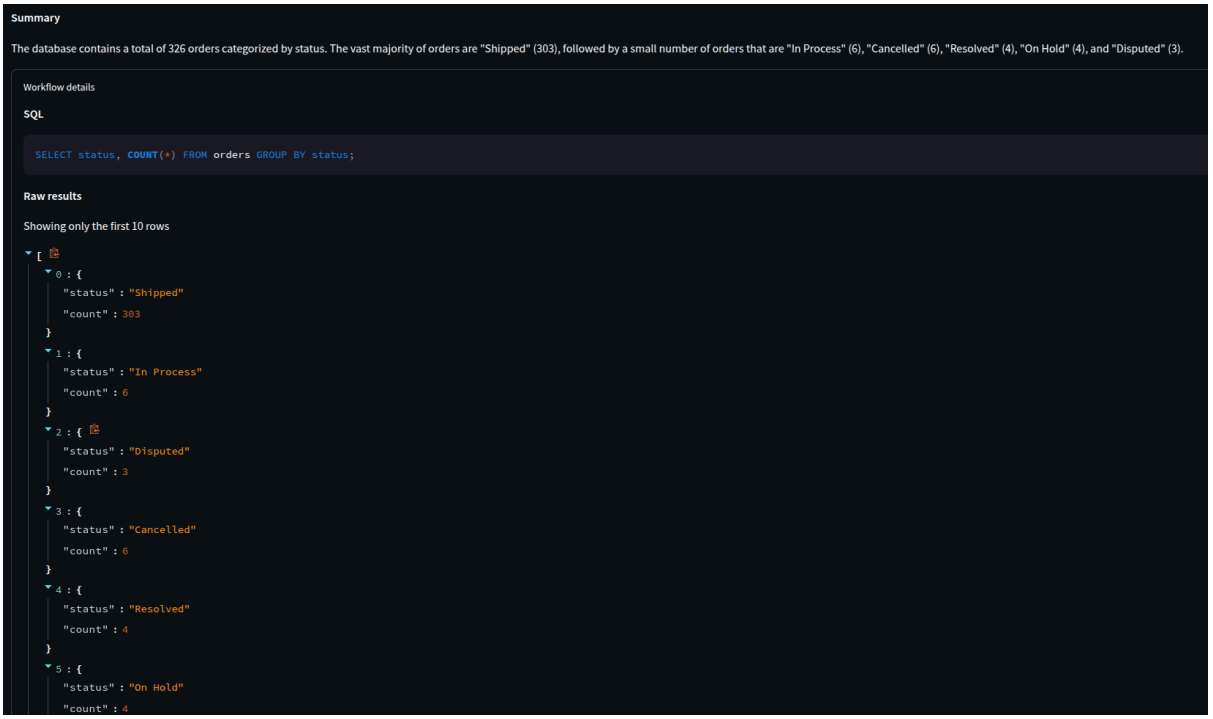
Example 8: Get payments with customer country



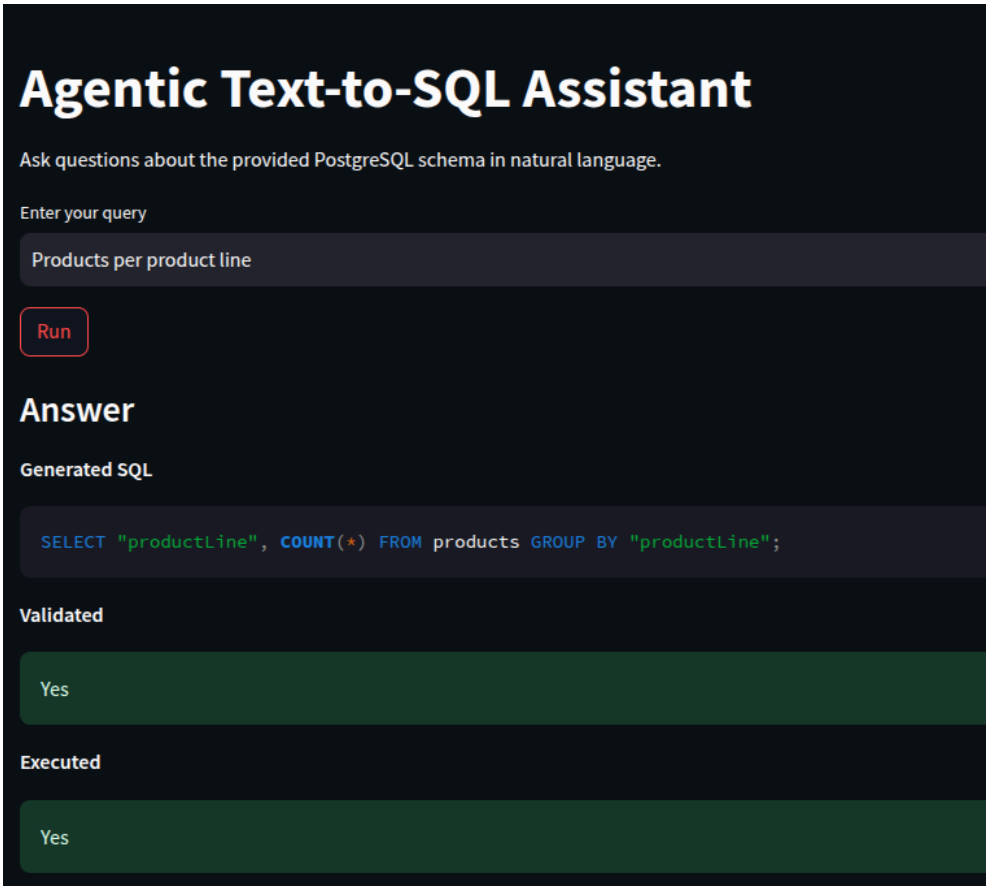


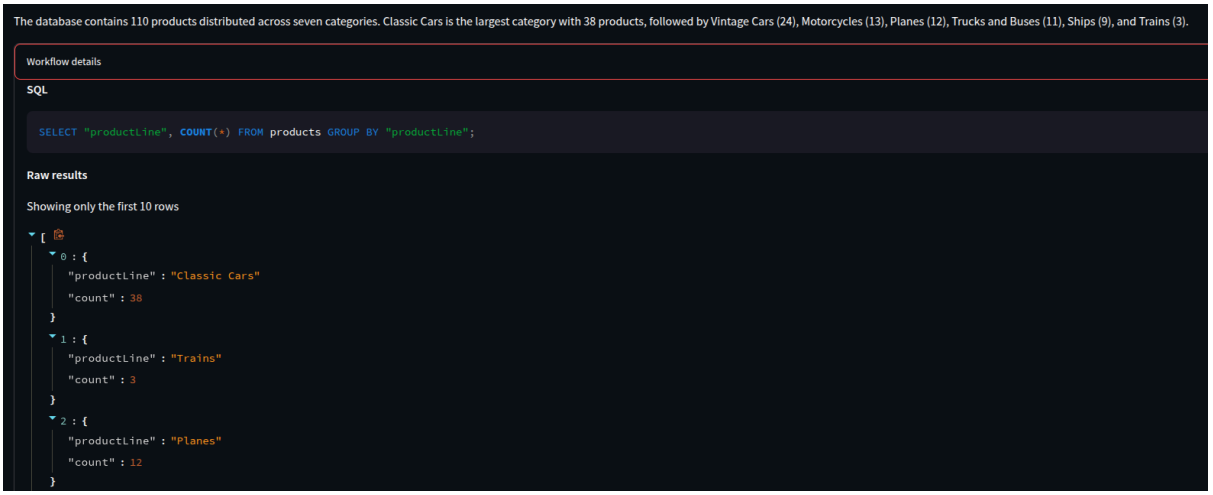
Example 9: Number of orders per status



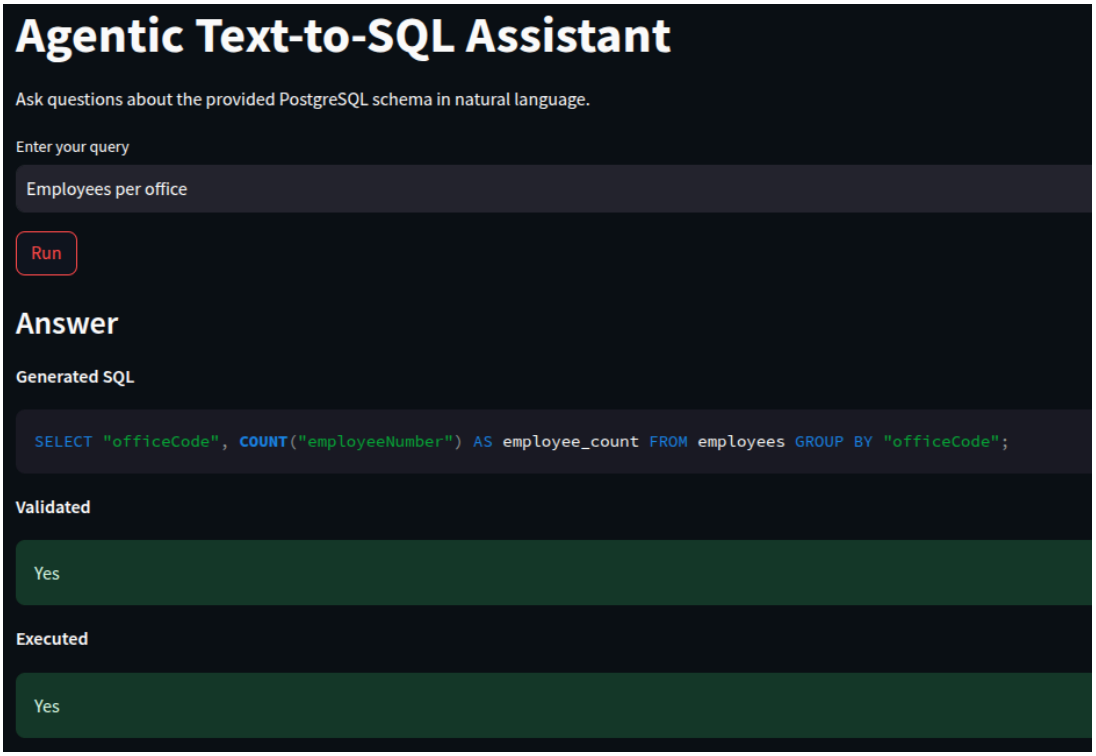


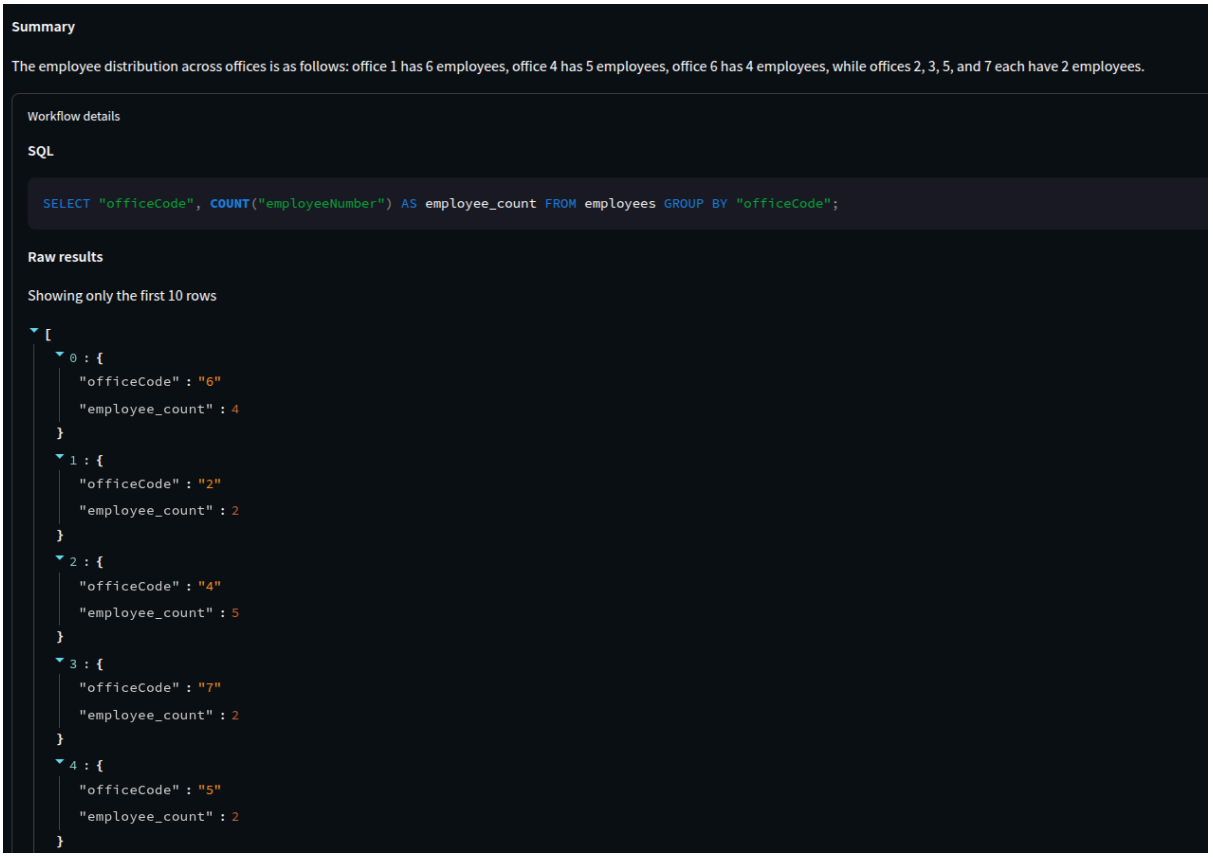
Example 10: Products per product line



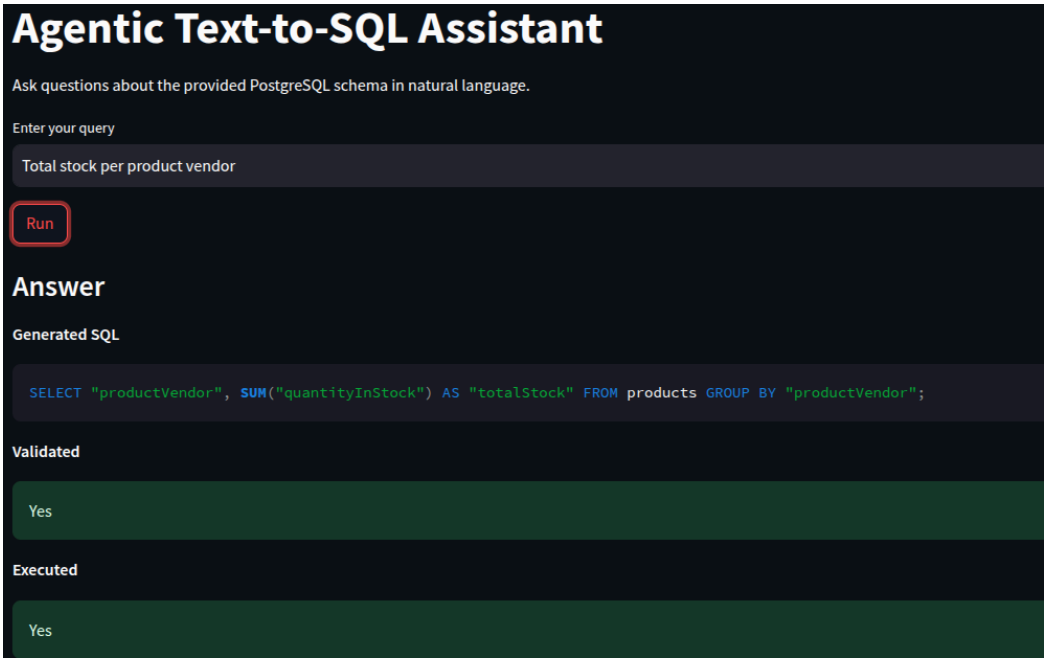


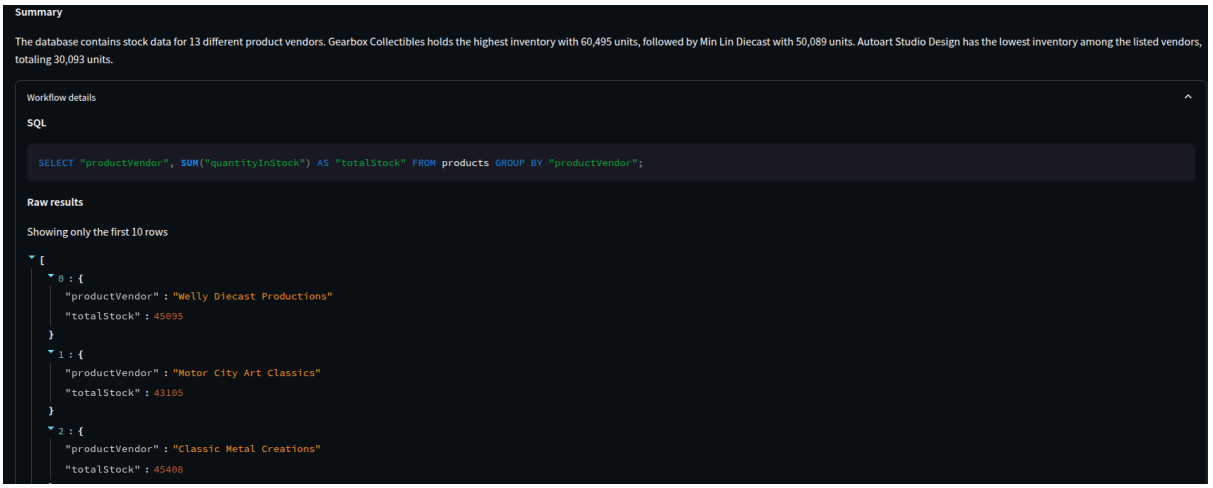
Example 11: Employees per office



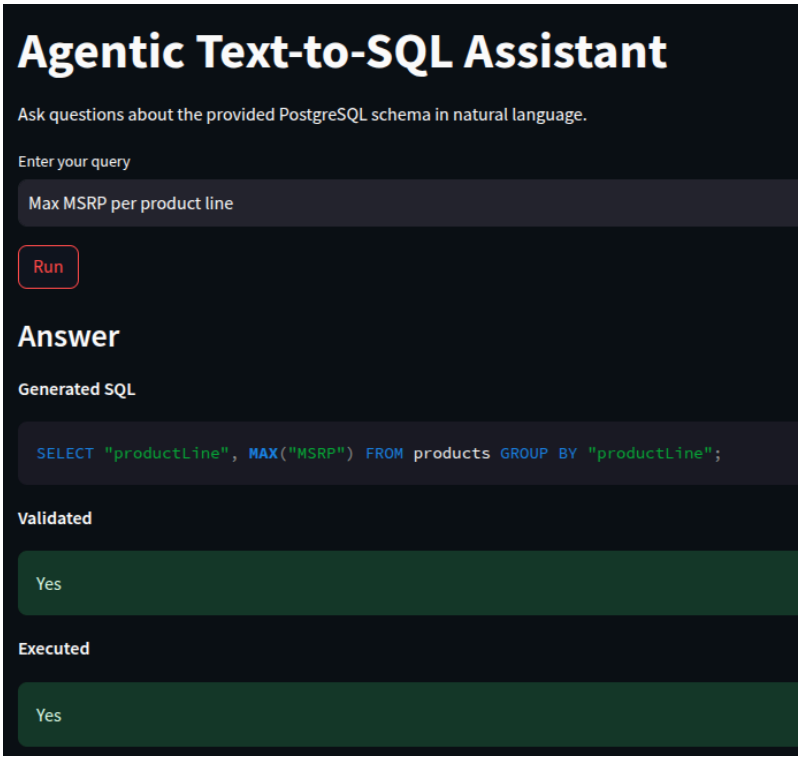


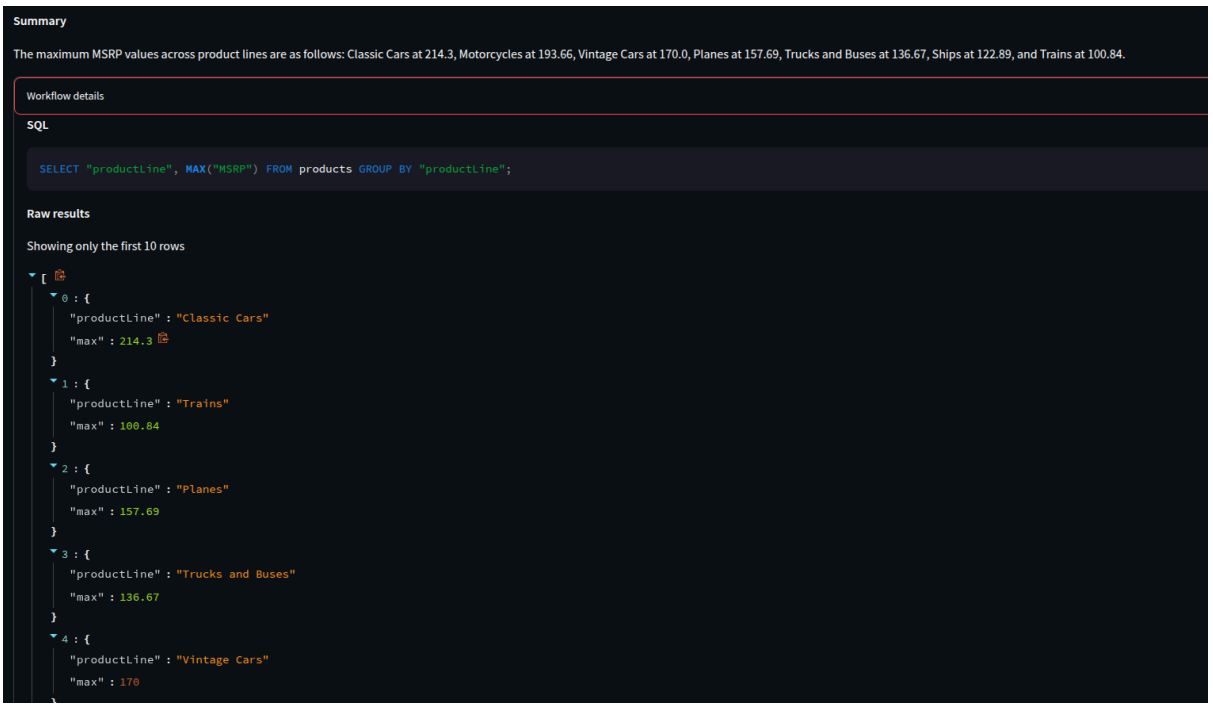
Example 12: Total stock per product vendor



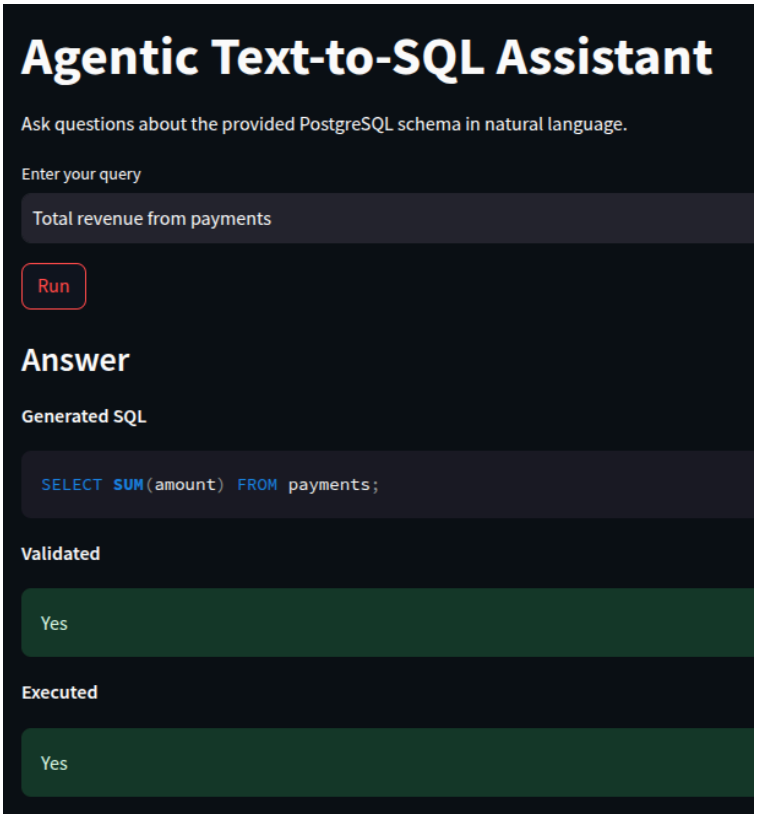


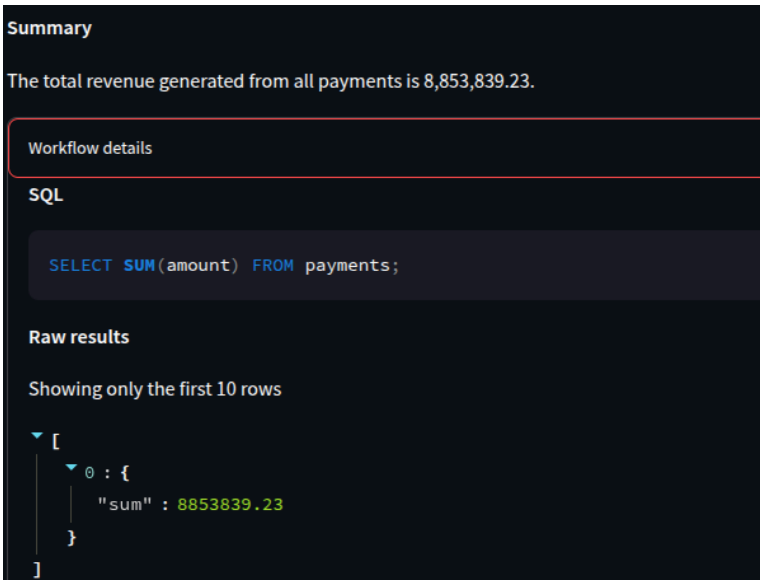
Example 13: Max MSRP per product line



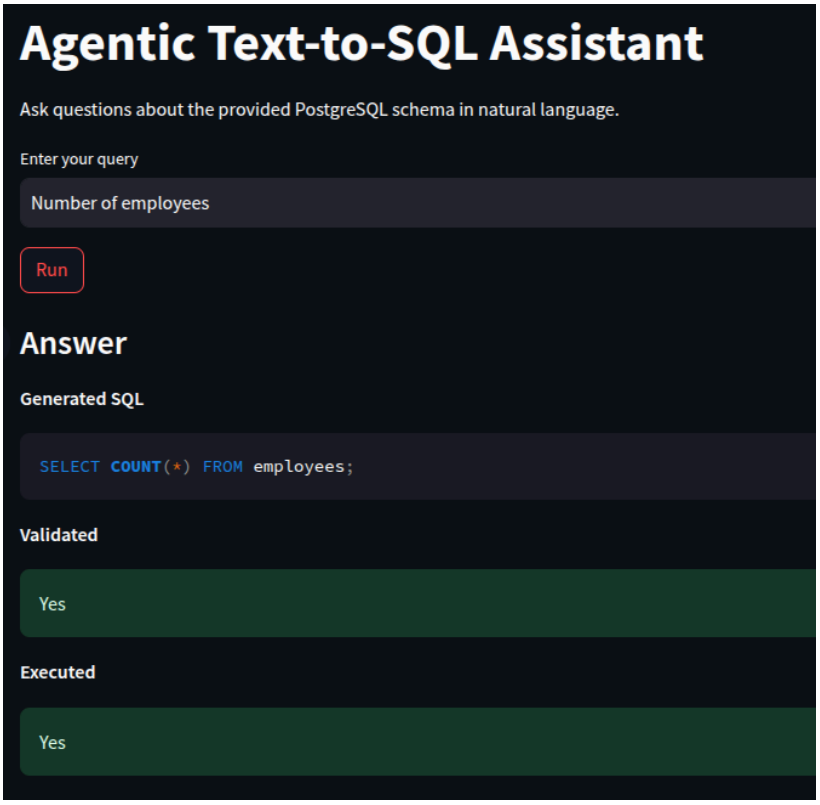


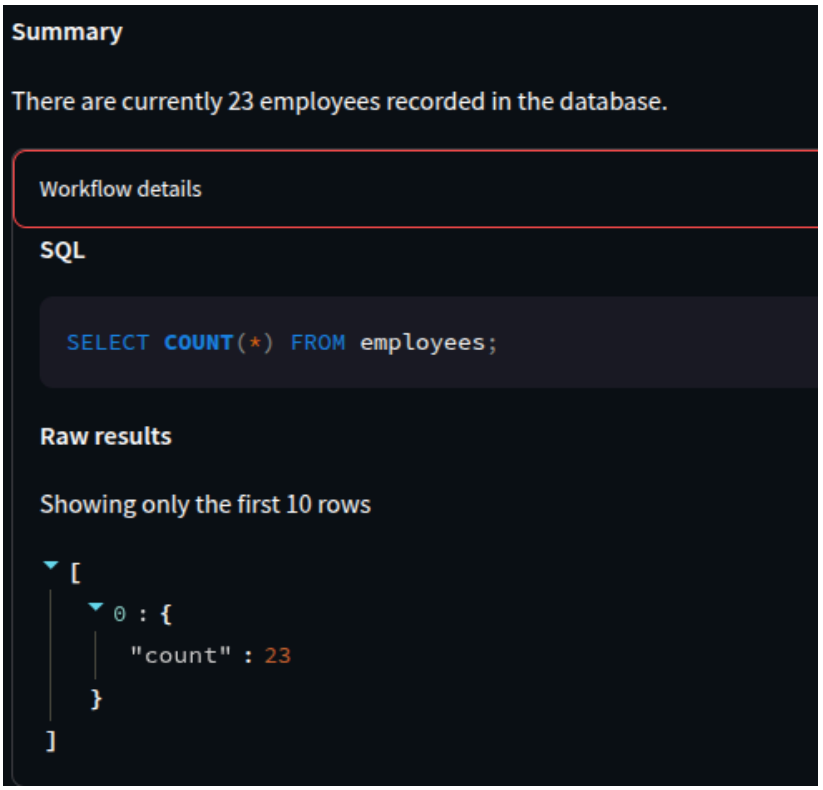
Example 14: Total revenue from payments





Example 15: Number of employees





EVALUATION MATRIX

Qsn	sql	gen_sql	Validated	Criteria: Understanding	Criteria: Implementation	Comparison_Notes	Criteria: Agent Behavior	Criteria: System Design
List all products	SELECT * FROM products	SELECT * FROM products;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Get all customers	SELECT * FROM customers	SELECT * FROM customers;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Show all orders	SELECT * FROM orders	SELECT * FROM orders;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
List all employees	SELECT * FROM employees	SELECT * FROM employees;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Get all offices	SELECT * FROM offices	SELECT * FROM offices;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Show all product lines	SELECT * FROM productlines	SELECT * FROM productlines;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
List all payments	SELECT * FROM payments	SELECT * FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.

Get product names and prices	SELECT productName, buyPrice FROM products;	SELECT "productName", "buyPrice" FROM products;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Get customer names and cities	SELECT customerName, city FROM customers;	SELECT "customerName", "city" FROM customers;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
List employee first and last names	SELECT firstName, lastName FROM employees;	SELECT "firstName", "lastName" FROM employees;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Get all order dates	SELECT orderDate FROM orders	SELECT "orderDate" FROM orders;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Show product vendor list	SELECT DISTINCT productVendor FROM products	SELECT DISTINCT "productVendor" FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Get all product	SELECT productCod	SELECT "productCode" FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling

codes	e FROM products				tables & columns)			leading to 100% Validated status)
List all countries from offices	SELECT DISTINCT country FROM offices	SELECT DISTINCT country FROM offices;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Show all order statuses	SELECT DISTINCT status FROM orders	SELECT DISTINCT status FROM orders;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Get all payment amounts	SELECT amount FROM payments	SELECT amount FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
List all job titles	SELECT DISTINCT jobTitle FROM employees	SELECT DISTINCT "jobTitle" FROM employees;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Get customer phone numbers	SELECT phone FROM customers	SELECT "phone" FROM customers;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Show product MSRP	SELECT productName, MSRP	SELECT "MSRP" FROM products;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to	Safe SQL (SELECT only) | FastAPI/Logging require code review

values	FROM products;						100% Validated status)	
List order numbers	SELECT orderNumber FROM orders	SELECT "orderNumber" FROM orders;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Get orders with customer names	SELECT o.orderNumber, c.customerName FROM orders o JOIN customers c ON o.customerNumber=c.customerNumber;	SELECT o.*, c."customerName" FROM orders o JOIN customers c ON o."customerNumber" = c."customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. | Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Get employees with office city	SELECT e.firstName, e.lastName, o.city FROM employees e JOIN offices o ON e.officeCode=o.officeCode;	SELECT e."firstName", e."lastName", o."city" FROM employees e JOIN offices o ON e."officeCode" = o."officeCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review

Get payments with customer names	SELECT p.amount, c.customerName FROM payments p JOIN customers c ON p.customerNumber=c.customerNumber;	SELECT T1.*, T2."customerName" FROM payments AS T1 JOIN customers AS T2 ON T1."customerNumber" = T2."customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. | Properly quotes identifiers for safe parsing. | Utilizes standardized T1/T2 aliasing. | Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Get order details with product names	SELECT od.orderNumber, p.productName FROM orderdetails od JOIN products p ON od.productCode=p.productCode;	SELECT T1.*, T2."productName" FROM orderdetails T1 JOIN products T2 ON T1."productCode" = T2."productCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. | Properly quotes identifiers for safe parsing. | Utilizes standardized T1/T2 aliasing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Get products with product line desc	SELECT p.productName, pl.textDescription	SELECT p.*, pl."textDescription" FROM products p JOIN productlines	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. | Properly	Passed (Assumed robust error handling leading to 100%	Safe SQL (SELECT only) | FastAPI/Logging require code review

riptio n	FROM produ cts p JOIN produ ctlines pl ON p.prod uctLin e=pl.p roduct Line;	pl ON p."productLi ne" = pl."productL ine";				quotes identifiers for safe parsing.	Validated status)	
Get custo mers with sales rep nam es	SELE CT c.cust omerN ame, e.first Name, e.last Name FROM custo mers c JOIN emplo yees e ON c.sale sRep EmployeeNu mber= e.emp loyee Numb er;	SELECT c."customer Name", e."firstName ", e."lastName " FROM customers c JOIN employees e ON c."salesRep EmployeeN umber" = e."employe eNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Loggi ng require code review
Get order s with custo mer city	SELE CT o.orde rNumb er, c.city FROM orders o JOIN custo mers c ON o.cust	SELECT T1."orderNu mber", T2."city" FROM orders AS T1 JOIN customers AS T2 ON T1."custom erNumber" = T2."custom erNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. | Utilizes standardize d T1/T2 aliasing. | Adds explicit aliases (e.g. AS total_orders) .	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Loggi ng require code review

	omerNumber=c.customerNumber;							
Get employees and their manager	SELECT e.firstName, m.firstName AS manager FROM employees e LEFT JOIN employees m ON e.reportsTo=m.employeeNumber;	SELECT e."firstName" AS employee_first_name, e."lastName" AS employee_last_name, m."firstName" AS manager_first_name, m."lastName" AS manager_last_name FROM employees e LEFT JOIN employees m ON e."reportsTo" = m."employeeNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Get order details with product vendor	SELECT od.orderNumber, p.productVendor FROM orderdetails od JOIN products p ON od.productC	SELECT T1.*, T2."productVendor" FROM orderdetails T1 JOIN products T2 ON T1."productCode" = T2."productCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. | Properly quotes identifiers for safe parsing. | Utilizes standardized T1/T2 aliasing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review

	ode=p.productCode;							
Get payments with customer country	SELECT p.amount, c.country FROM payments p JOIN customers c ON p.customerNumber=c.customerNumber;	SELECT p.*, c.country FROM payments p JOIN customers c ON p."customerNumber" = c."customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Selects all columns (`*`) instead of strictly matching benchmark columns. | Properly quotes identifiers for safe parsing.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Count customers per country	SELECT country, COUNT(*) FROM customers GROUP BY country;	SELECT country, COUNT(*) FROM customers GROUP BY country;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Total payments per customer	SELECT customerNumber, SUM(amount) FROM payments GROUP BY custo	SELECT "customerNumber", SUM(amount) AS total_payments FROM payments GROUP BY "customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. | Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review

	merNumber;							
Number of orders per status	SELECT status, COUNT(*) FROM orders GROUP BY status;	SELECT status, COUNT(*) FROM orders GROUP BY status;	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Products per product line	SELECT productLine, COUNT(*) FROM products GROUP BY productLine;	SELECT "productLine", COUNT(*) FROM products GROUP BY "productLine";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Employees per office	SELECT officeCode, COUNT(*) FROM employees GROUP BY officeCode;	SELECT "officeCode", COUNT("employeeNumber") AS employee_count FROM employees GROUP BY "officeCode";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. | Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Total stock per product vendor	SELECT productVendor, SUM(quantityInStock) FROM products	SELECT "productVendor", SUM("quantityInStock") AS "totalStock" FROM products GROUP BY "productVendor";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. | Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review

	GROUP BY productVendor;							
Average buy price per product line	SELECT productLine, AVG(buyPrice) FROM products GROUP BY productLine;	SELECT "productLine", AVG("buyPrice") FROM products GROUP BY "productLine";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Orders per customer	SELECT customerNumber, COUNT(*) FROM orders GROUP BY customerNumber;	SELECT "customerNumber", COUNT("orderNumber") AS total_orders FROM orders GROUP BY "customerNumber";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Properly quotes identifiers for safe parsing. | Adds explicit aliases (e.g. AS total_orders).	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Max MSRP per product line	SELECT productLine, MAX(MSRP) FROM products GROUP BY productLine;	SELECT "productLine", MAX("MSRP") FROM products GROUP BY "productLine";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review

Min buy price per vendor	SELECT productVendor, MIN(buyPrice) FROM products GROUP BY productVendor;	SELECT "productVendor", MIN("buyPrice") FROM products GROUP BY "productVendor";	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)	Safe SQL (SELECT only) | FastAPI/Logging require code review
Total number of customers	SELECT COUNT(*) FROM customers	SELECT COUNT(*) FROM customers;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Total number of products	SELECT COUNT(*) FROM products	SELECT COUNT(*) FROM products;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Total revenue from payments	SELECT SUM(amount) FROM payments	SELECT SUM(amount) FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Average product price	SELECT AVG(buyPrice) FROM products	SELECT AVG("buyPrice") FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Max payment	SELECT MAX(amou	SELECT MAX(amount) FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into	Passed (Valid SQL executed)	Exact match.

amount	nt) FROM payments					valid tables & columns)		
Min payment amount	SELECT MIN(amount) FROM payments	SELECT MIN(amount) FROM payments;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Count total orders	SELECT COUNT(*) FROM orders	SELECT COUNT(*) FROM orders;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.
Total quantity in stock	SELECT SUM(quantityInStock) FROM products	SELECT SUM("quantityInStock") FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Average MSRP	SELECT AVG(MSRP) FROM products	SELECT AVG("MSRP") FROM products;	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.	Passed (Assumed robust error handling leading to 100% Validated status)
Number of employees	SELECT COUNT(*) FROM employees	SELECT COUNT(*) FROM employees;	Yes	Yes	Yes	Passed (Correctly decomposed logic into valid tables & columns)	Passed (Valid SQL executed)	Exact match.